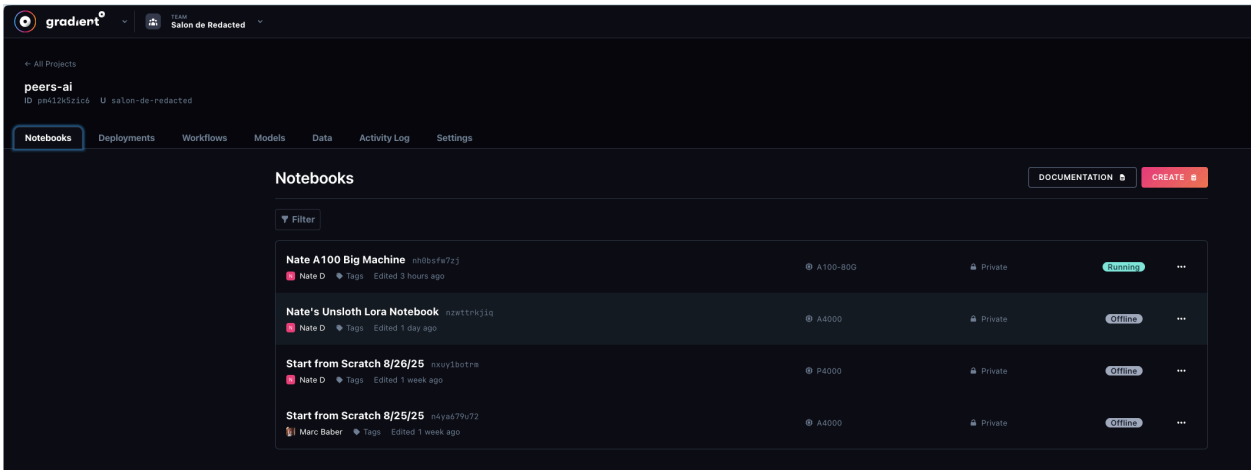
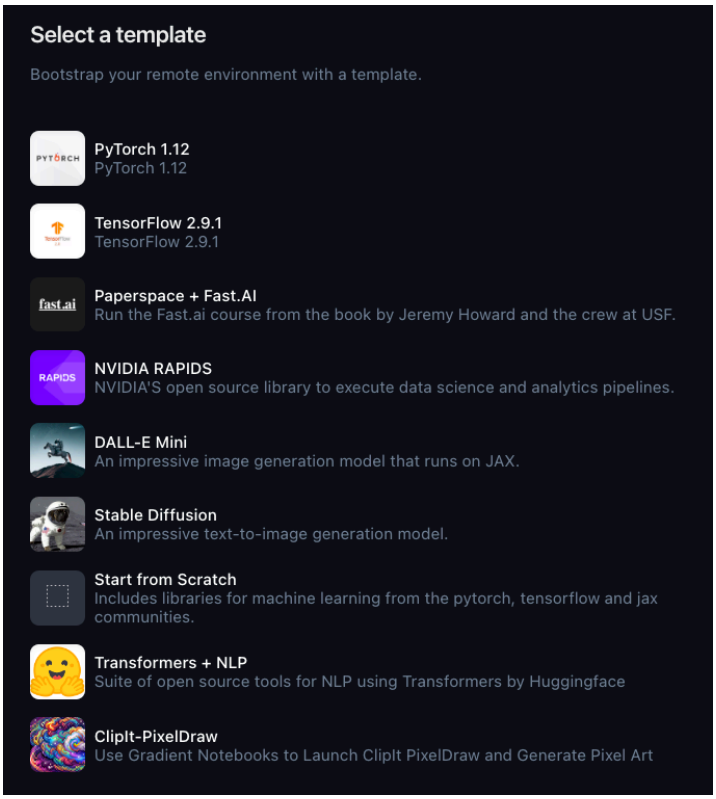
PEERS Paperspace Training and Inference Guide

Part 1: How to train a LORA model

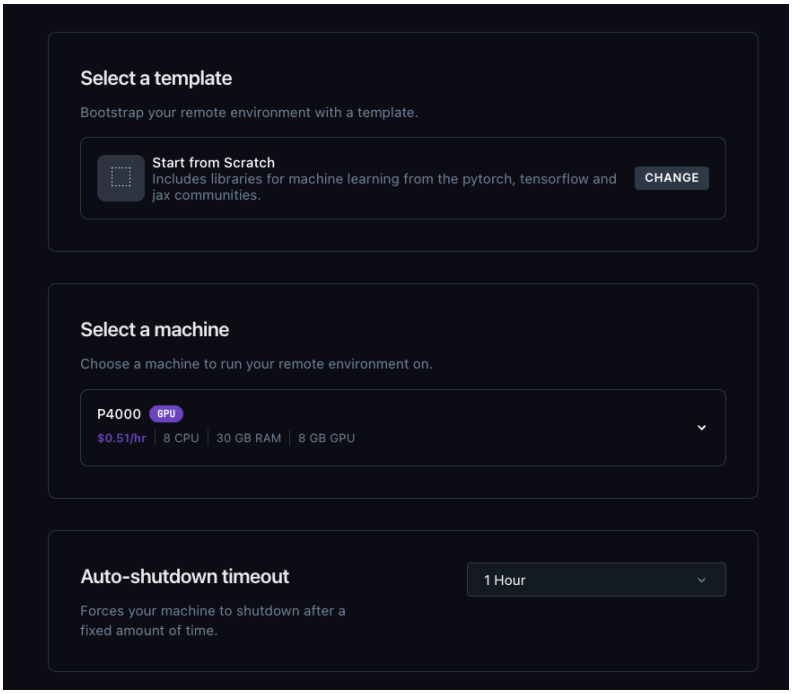
1. Navigate to the Salon de Redacted project here:
https://console.paperspace.com/salon-de-redacted/projects/pm412k5zic6/notebooks



2. You will see a list of templates. Click “Start from Scratch”.



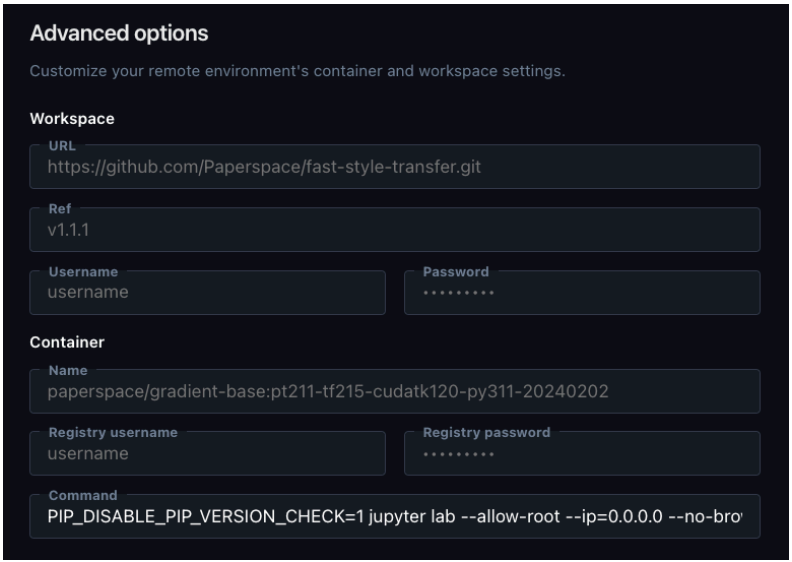
3. You will now select a machine by clicking the “Select a machine” option.



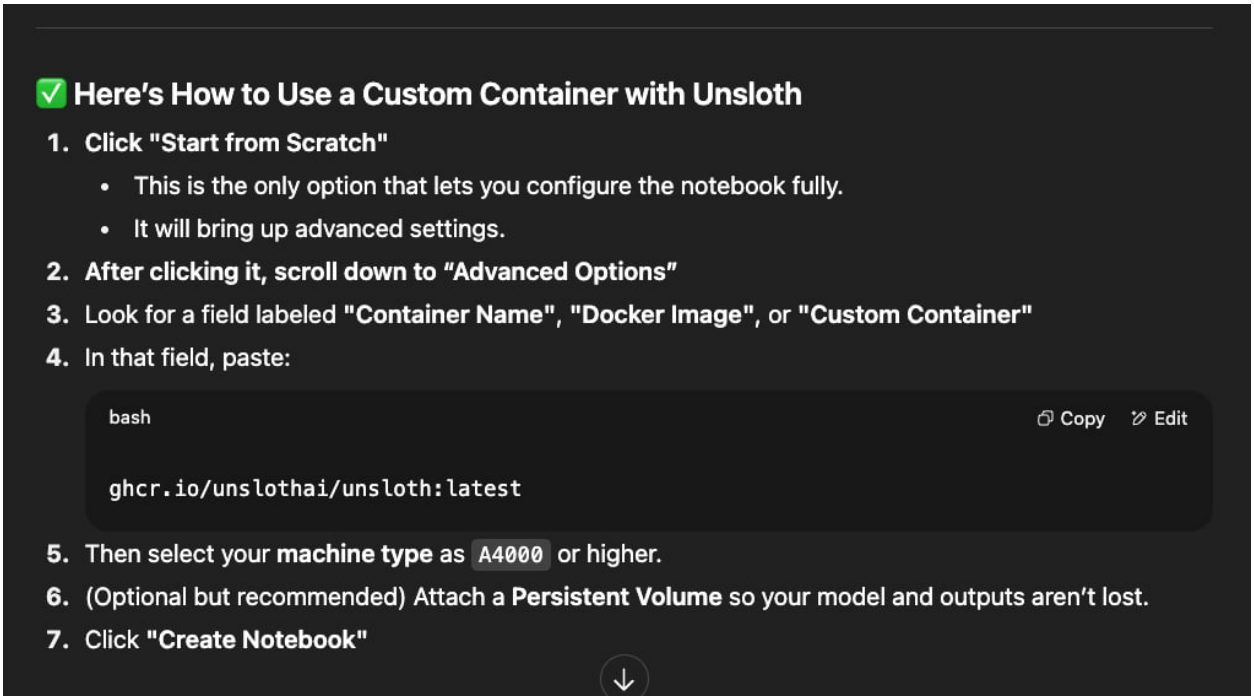
You will want at least an A5000 for training a 7B parameter model, and at least an A100 for training a 32B parameter model.

4. Now you will want to set an Auto-shutdown timeout. If you plan to train 32B models, then you may want to select a week timeframe.

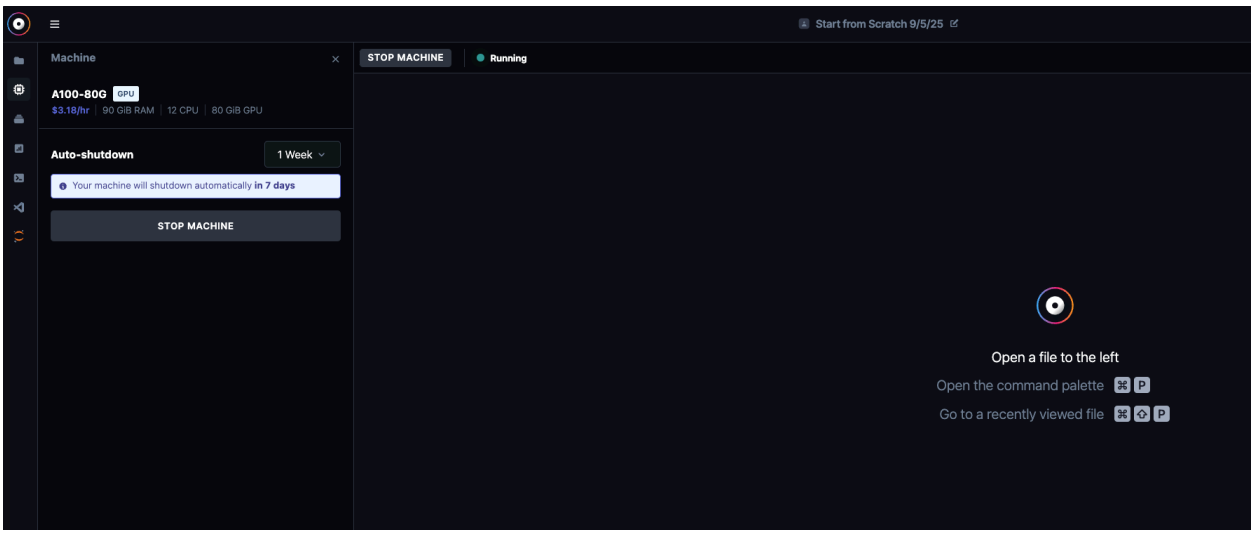
5. Now click “Advanced options”.



6. We will use a custom container. Use the below settings and type them in.



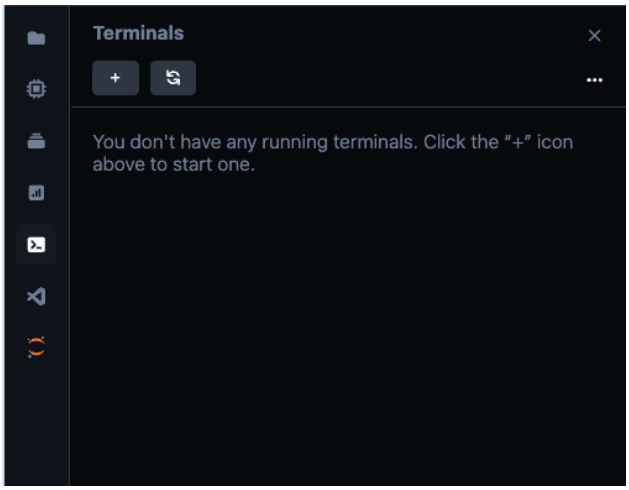
7. Then click "Start Notebook". Then you will have to wait a few minutes for the machine to start up. When it does, you will see the UI below.



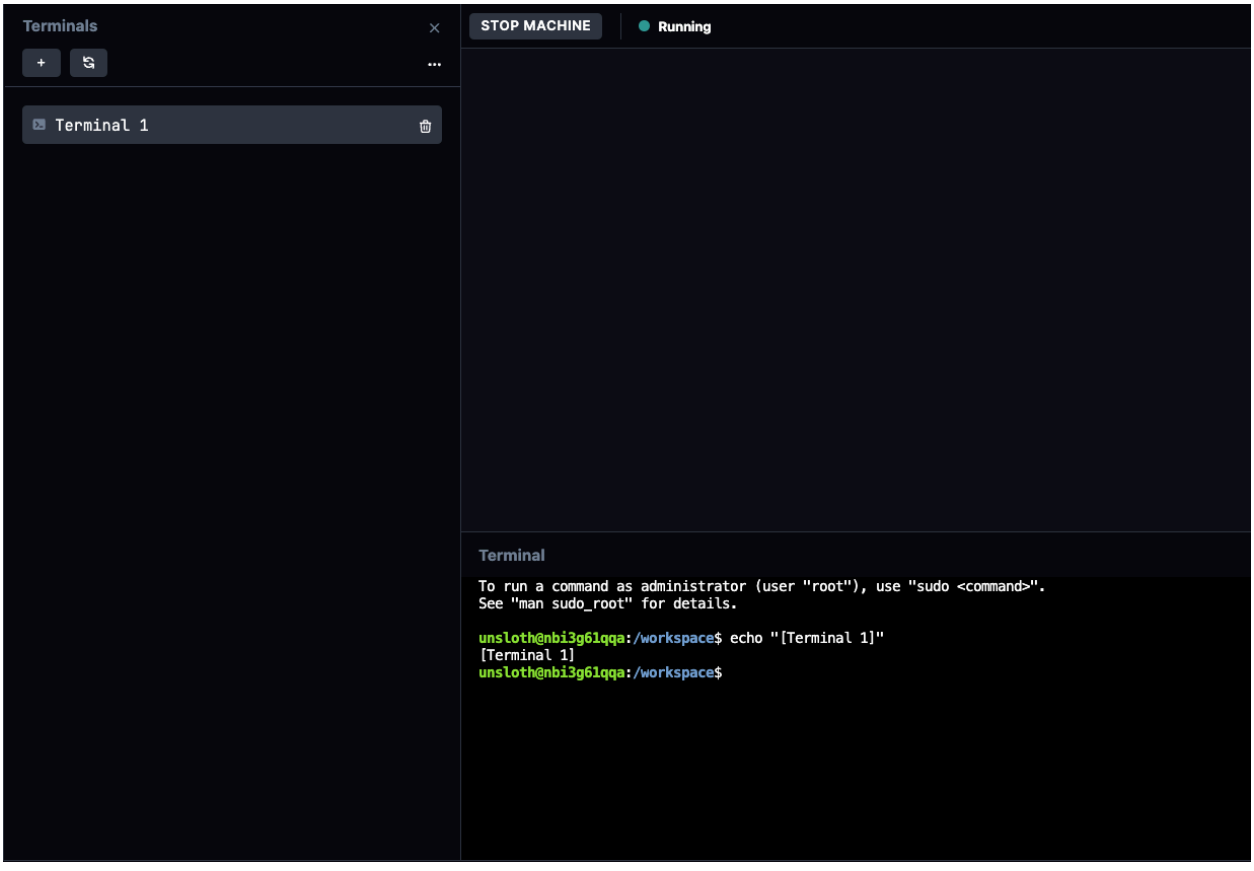
On the top left, you will see a series of icons that allow you to work with your machine. Respectively, they are:

- Folder icon: show your workspace files
- Computer chip icon: show machine details
- Stack blocks icon: show your uploaded datasets
- Graph icon: show cpu and machine usage
- Command-line icon: allows you to create a terminal and switch between those that are open

8. For now, let's open up a terminal. Click the command-line icon and then click the "+" button under Terminals.



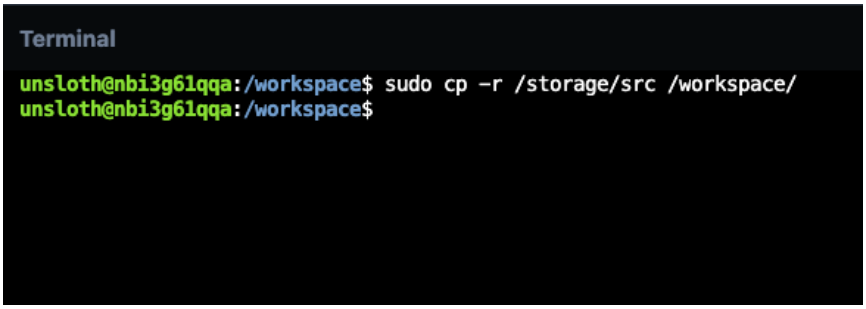
You should now have a Linux terminal interface.



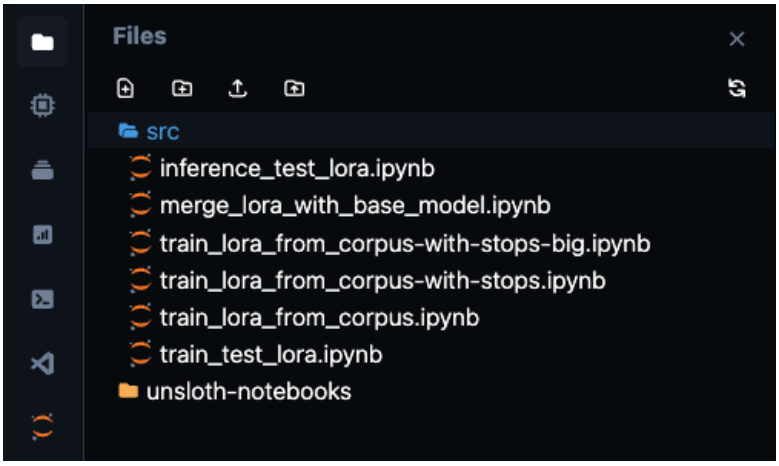
9. Now we are going to copy the training AI source files from storage to our workspace.

A quick explanation of the filesystem setup:
* /workspace - this contains the current workspace for the machine. This workspace is temporary and will only last while the machine is started.
* /storage - this is persistent storage that persists across notebooks and across machine restarts. Important files are stored here and copied to the workspace.
* /datasets - this contains any mounted datasets

To perform the copy, do the following. You will need to use the sudo password of “unsloth”.



10. Now click the Folder icon and then click the refresh to make sure the new src folder is visible.

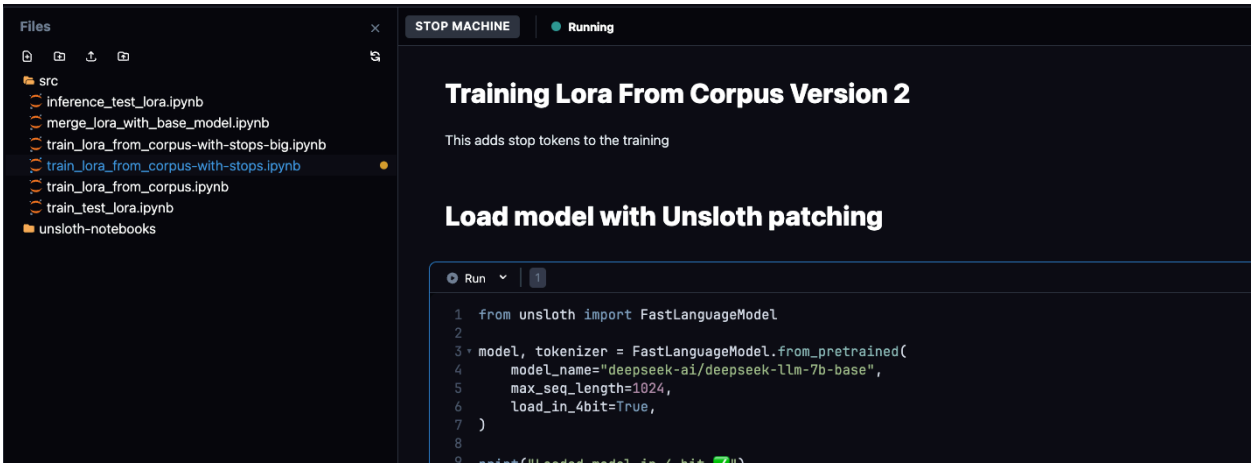


11. Now you will select either “train_lora_from_corpus-with-stops.ipynb” or “train_lora_from_corpus-with-stops-big.ipynb”. Use the former for 7B models or the latter for 32B models.

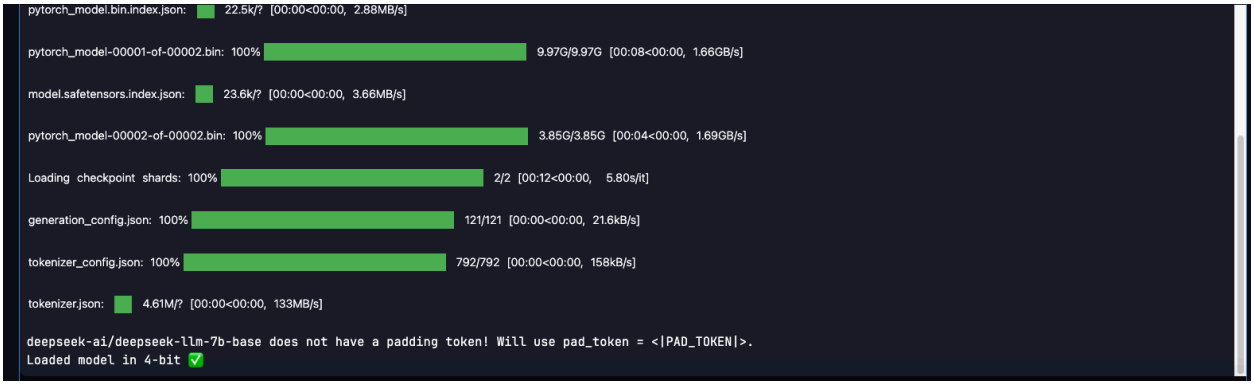
In this document, I’ll show you how to train a 7B model so use the former.

12. Now you should be able to run the various parts of the AI training script. First, let's run the script under "Load model with Unsloth patching." Click Run here.

Optionally, you can change the model to something else. Here we are using Deepseek 7B.



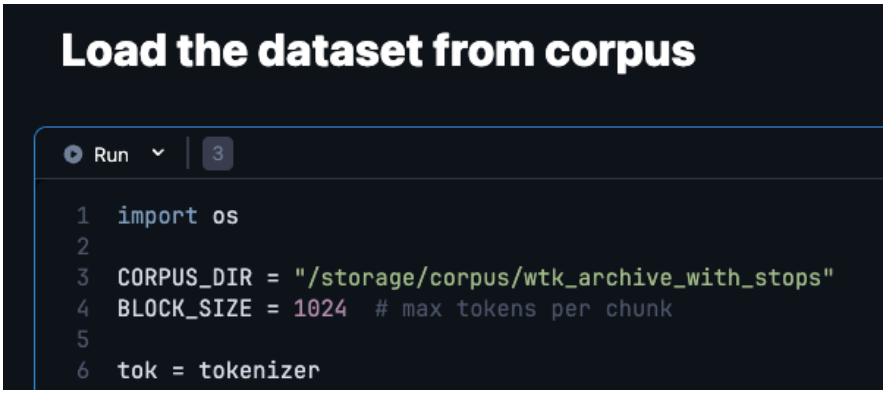
When complete, you should see this output.



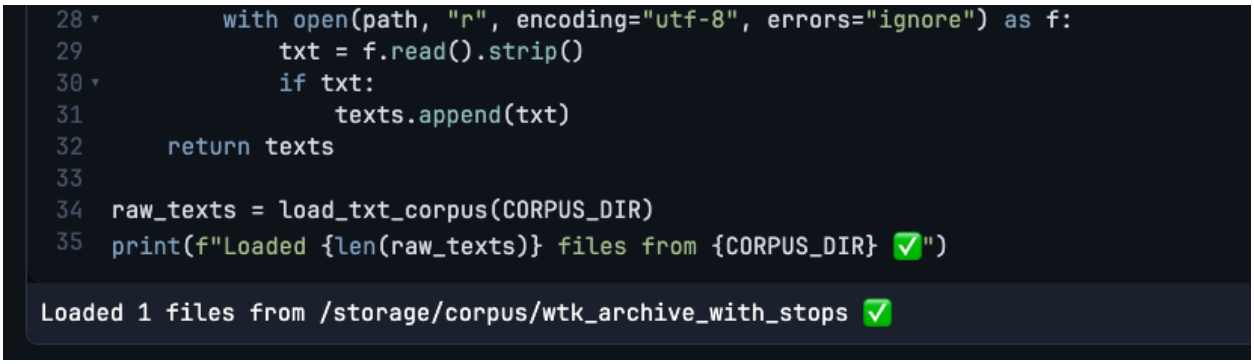
13. Next run the "Apply LoRa adapter" script.

14. Next run the “Load the dataset from corpus” but first make sure the CORPUS_DIR is set appropriately to the corpus inside “storage/“.

For training the 7B model, using the wtk_archive_with_stops corpus contains the want to know data and, since it is small at 22 MB, is suitable to train a first model on.



If it works correctly, then you should see the output that shows the number of loaded files.



15. Next run “Tokenize with EOS appended” script. If it works correctly, then you should see the number of packed training chunks.

15. Now let's create the model output directory in /storage so that it will persist past machine shutdowns.

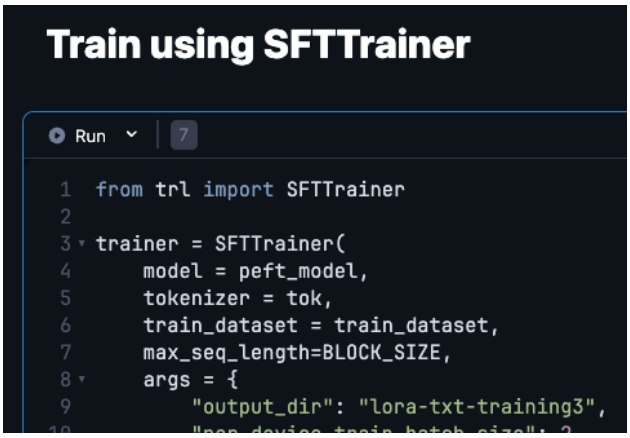
Use the following commands to do so (replace with your own model name):

```
unsloth@nvsz1xm1yp:/$ cd /storage/models
unsloth@nvsz1xm1yp:/storage/models$ sudo mkdir my_test_model_name
unsloth@nvsz1xm1yp:/storage/models$ sudo chmod 775 ./my_test_model_name/
```

Note: the *sudo* is important, because normally root owns these directories. If you ever run into permissioning errors, then use *sudo.*

Note: the *sudo password* is *'unsloth'.*

16. Next run "Train using SFTTrainer" but first make sure the output directory for the model training matches the save locations.



This will run until all training iterations complete. If successful, then you will both the "Training complete" and "Training results saved" messages.

Your trained model should be in the output directory you selected.

17. Optional step, if you want to push your model to Huggingface, from the terminal (you can open up using the command icons on the top-left), type "huggingface-cli login".

```
unsloth@nvsz1xm1yp:/storage/src$ huggingface-cli login
⚠  Warning: 'huggingface-cli login' is deprecated. Use 'hf auth login' instead.

    To log in, `huggingface_hub` requires a token generated from https://huggingface.co/settings/tokens .
Enter your token (input will not be visible):
Add token as git credential? (Y/n) Y
Token is valid (permission: fineGrained).
The token `peers-testing2` has been saved to /workspace/.cache/huggingface/stored_tokens
```

You will be queried for the Huggingface login token. Copy the below and paste it into the terminal:
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

Select ‘Y’ when asked if you want to add the token as a Git credential.

You should have the permissions now to push to Huggingface.

18. After completing the previous step, run the below snippet:

Push LoRa to Huggingface

Run | 1

```python
from huggingface_hub import HfApi, upload_folder

repo_id = "peers-ai/" + OUTPUT_DIR
folder = OUTPUT_DIR  # contains adapter_config.json & adapter_model.bin

api = HfApi()
# create the repo if it doesn't exist
api.create_repo(repo_id, repo_type="model", private=True, exist_ok=True)

# upload all files in the folder
upload_folder(
    repo_id=repo_id,
    folder_path=folder,
    repo_type="model",
)
print(f"✅ Uploaded to https://huggingface.co/{repo_id}")
```

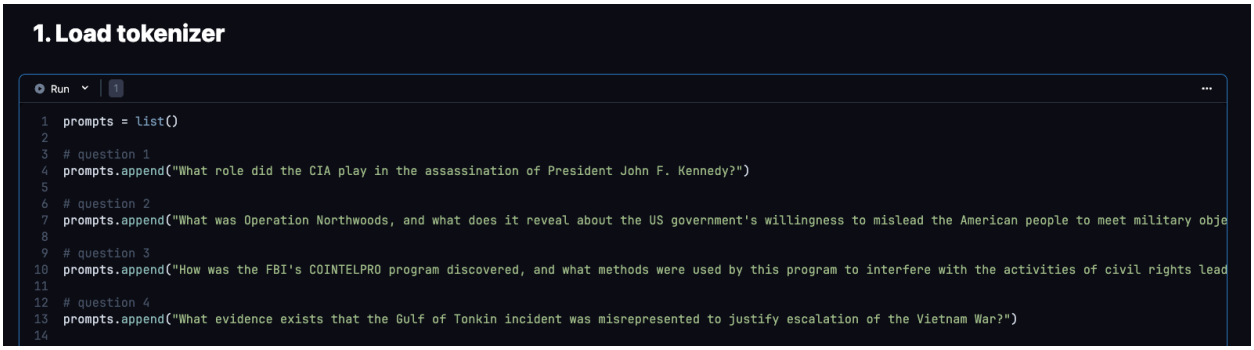
Your model should then be uploaded to Huggingface and you should see the below output.

```
Processing Files (0 / 0)                : |          |  0.00B /  0.00B
New Data Upload                         : |          |  0.00B /  0.00B
  ...training2/adapter_model.safetensors:  97%|#########7| 15.3MB / 15.7MB
✅ Uploaded to https://huggingface.co/peers-ai/deepseek-7b-my-lora1-with-stops
```

Part 2: How to Do Inference on a Local Model

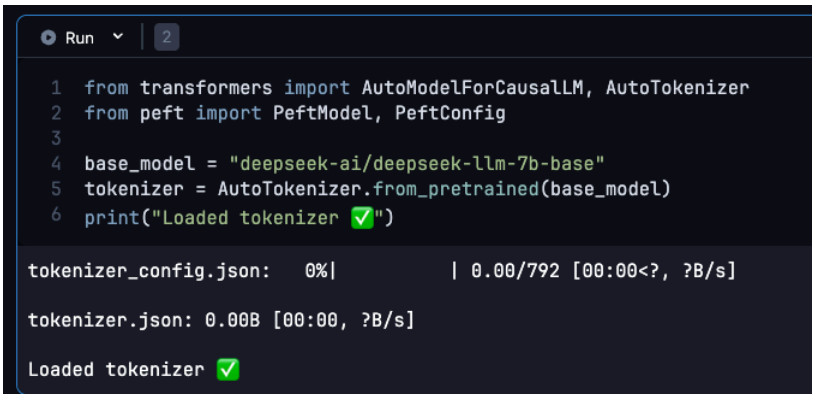
Doing inference is easier than doing training. Here we will run the standardized prompt test on a local model that is present on Paperspace storage.

1. Open the file “inference_test_lora.ipynb” and then run the Load Prompts snippet



This loads the prompts we will use for our test.

2. Run the Load Tokenizer snippet



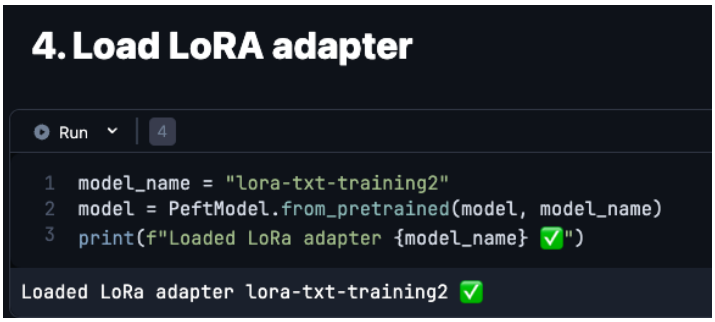
3. Run the Load base model

You have the option of loading in 4 bit (use load_in_4bit=True) or 8 bit ((use load_in_8bit=True).

Important note – the base model must be the same as the one you trained the Lora with.

4. Run Load the Lora adapter

Here you will need to point model_name to the directory of the model you just trained (likely in /storage/models/).



5. Run inference (either with stops or without)

Here you will have the choice to run either 5a (no stops) or 5b (with stops).

The difference between the two versions is what happens when a stop token is reached. The former will keep going. The latter will stop immediately. Oftentimes, the output after a stop token won't pertain to the topic at hand.

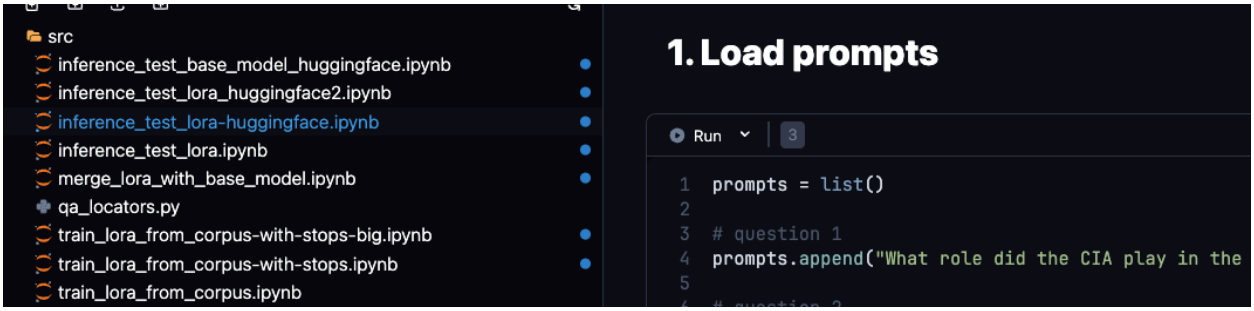
Part 3: How to Do Inference with a Huggingface Lora

This is similar to the previous but we will now use one of our trained Loras that we have pushed to Huggingface.

You can see your uploaded models here.
https://huggingface.co/peers-ai

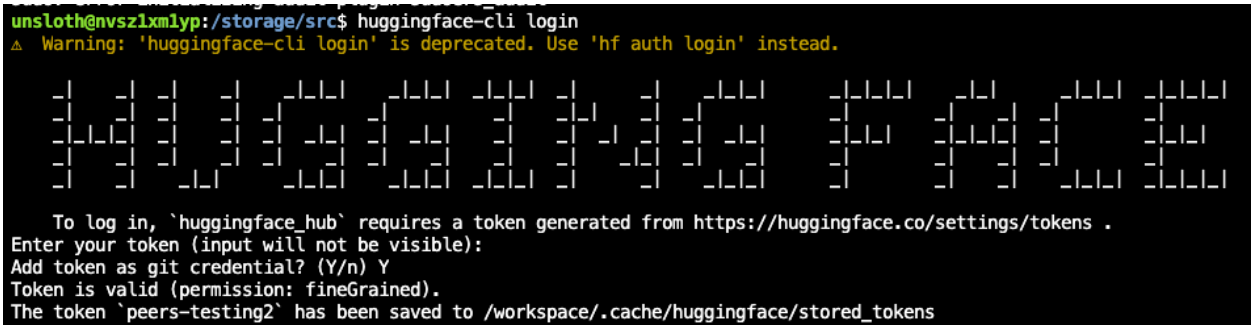
For the example here, I will use the below model:
https://huggingface.co/peers-ai/deepseek-7b-my-lora1

1. Open the file "inference_test_lora-huggingface.ipynb" and run steps 1-3 just like the previous



2. Now you will need to login into Huggingface using the huggingface-cli tool.

See Part 1-17 for the instructions to do this.



3. Now the difference comes in step 4 of “Load Lora Adapter from Huggingface”

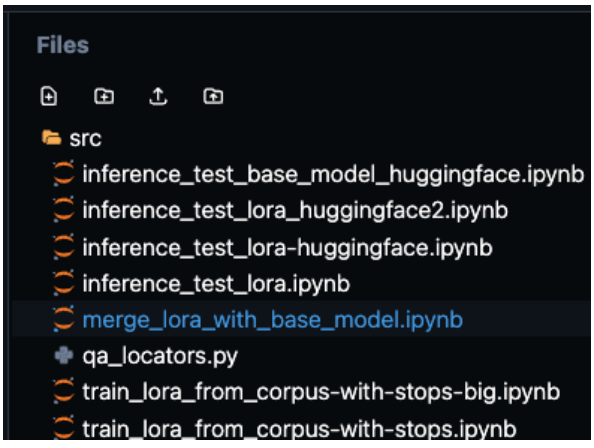
Change the variable huggingface_lora_name to the appropriate lora model from Huggingface and then run the snippet.

4. Now you can do inference just like in Step 5 of the previous section.

Part 3: How to Merge Lora with Base Model

The following shows to merge a Lora you have pushed to Huggingface with a base model. You will end up with a single model that has been updated and which you can work with locally or on the deployment solution.

1) Open the file “merge_lora_with_base_model.ipynb"



2) Update the “Imports & basic config” section and then run

Update BASE_ID to use the name of the base model on Huggingface.

Update ADAPTER_ID to use the name of your Lora in the peers-ai account on Huggingface.
Update OUT_DIR to a local place on the workspace (in /workspace) or inside our Paperspace storage (in /storage/models/)
Update PUSH_TO to specify the Huggingface name of the new merged model and lora for pushing to Huggingface.

Note due to permissioning issues, you might have to create the new out directory first inside of /storage/models and do chmod 775 to give it appropriate permissions.

1) Imports & basic config

Run | 3

```
import os, torch, shutil
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import PeftModel
from huggingface_hub import login, HfApi

# ---- Edit these IDs as needed ----
BASE_ID    = "deepseek-ai/deepseek-r1-distill-qwen-32b"          # base model
ADAPTER_ID = "peers-ai/deepseek-32b-my-lora1-with-stops"          # your LoRA repo
OUT_DIR    = "./merged-deepseek-32b-with-stops"                   # where to save merged
PUSH_TO    = "peers-ai/deepseek-32b-my-lora1-with-stops-merged"  # e.g., "yourname/merged-deepseek-32b-with-stops" or leave "" to skip pushing

# dtype options: "auto", "bfloat16", "float16", "float32"
```

2) Log into Hugging using snippet “Login to Huggingface” or use the huggingface-cli tool

2) Login to Huggingface

Run | 4

```
import os

# Only needed if your base/adapter are gated/private or if you'll push to Hub.
HF_TOKEN = os.getenv("HF_TOKEN", "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")  # set in environment or paste here
if HF_TOKEN:
    login(token=HF_TOKEN)
    print("Login success")
else:
    print("HF_TOKEN not set—continuing without Hub login.")
```

Note you might have to update the token here.

If you want to use the huggingface-cli tool, then see Part 1-17 for instructions.

3) Run the “Load Tokenizer” snippet to load the base model tokenizer

3) Load tokenizer

Run | 5

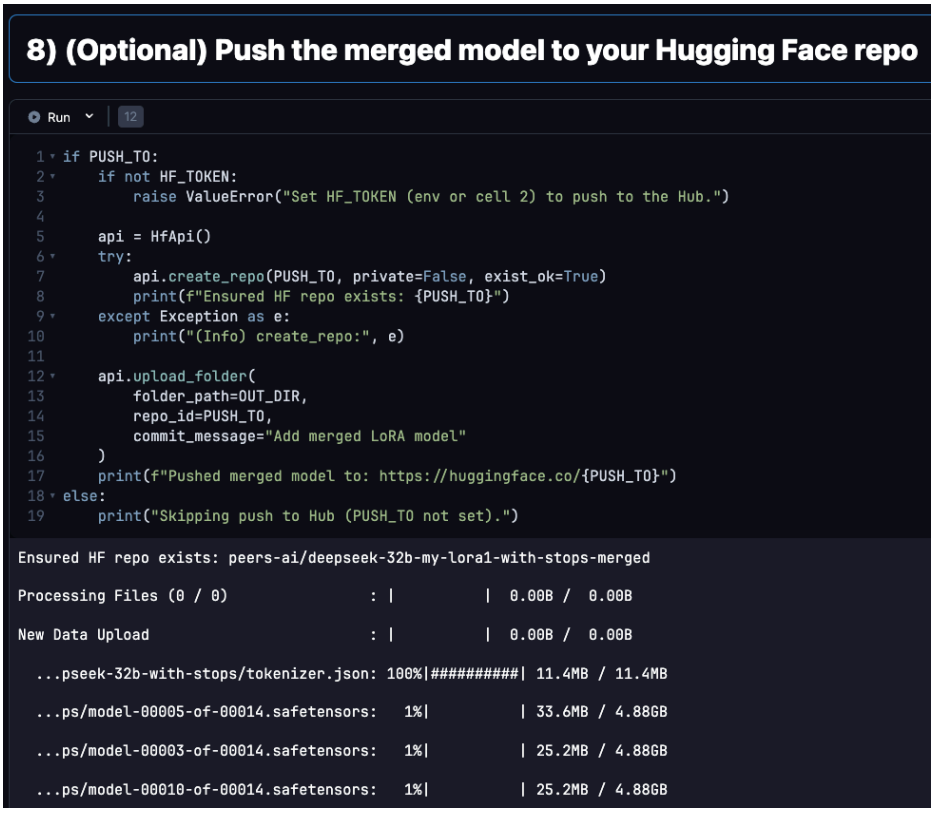
```
tok = AutoTokenizer.from_pretrained(
    BASE_ID,
    use_fast=True,
    trust_remote_code=TRUST_REMOTE_CODE
)
print("Loaded tokenizer from:", BASE_ID)

tokenizer_config.json: 0.00B [00:00, ?B/s]

tokenizer.json: 0.00B [00:00, ?B/s]

Loaded tokenizer from: deepseek-ai/deepseek-r1-distill-qwen-32b
```

4) Run the snippet “Load base model”

As the name suggests, you will have the option to load in 4 bit or 8 bit.

5) Now run both steps 6+7 to do the model merge and save locally (this might take a while)

6) Optional, you can run the “Push the merged model to Huggingface” snippet to save the new merged model on Huggingface



If it works, then you will see it upload to Huggingface.